Actividad 02

Tema: Teorema maestro y relaciones de recurrencia

Nota

Estudiante	Escuela	Asignatura
Capatinta Almanza Camilo B. A. ccapatinta@unsa.edu.pe	Escuela Profesional de Ingeniería de Sistemas	Análisis y Diseño de Algoritmos Semestre: IV Código: 1702231

Actividad	Tema	Duración
02	Teorema maestro y relaciones de recurrencia	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - B	Del 17 septiembre 2026	Al 20 septiembre 2026

Índice

1. Teorema Maestro y Relaciones de Recurrencia **3**

2. Teorema maestro **3**
2.1. Objetivo . . . 3
2.2. Relaciones de recurrencia y divide y vencerás . . . 3
2.3. Forma general de la recurrencia . . . 3
2.4. Intuición: el árbol de recursión . . . 4
2.5. Enunciado del Teorema Maestro . . . 4
2.6. Procedimiento para aplicar el teorema . . . 5
2.7. Limitaciones del teorema . . . 5
2.8. Tres casos de uso . . . 5
2.9. Ejemplo adicional: el caso 3 . . . 6
2.10. Resumen de los casos de uso . . . 7

3. Relaciones de Recurrencia **7**
3.1. Objetivo . . . 7
3.2. Definición . . . 7
3.3. Recurrencias en el análisis de algoritmos . . . 7
3.4. Clasificación de las recurrencias . . . 8
3.5. Métodos de resolución . . . 8
3.6. Casos de uso . . . 9
3.7. Ejemplos adicionales por método . . . 11
3.8. Resumen . . . 12

1. Teorema Maestro y Relaciones de Recurrencia

Investigar el Teorema Maestro como herramienta para resolver relaciones de recurrencia de la forma $T(n) = a\,T(n/b) + f(n)$, con $a \geq 1$ y $b > 1$.

- Indicar los casos de análisis que contempla el teorema, según la comparación entre $f(n)$ y $n^{\log_b a}$.
- Proponer tres casos de uso, planteando la relación de recurrencia de cada uno y resolviéndola con el teorema.

2. Teorema maestro

2.1. Objetivo

Investigar el Teorema Maestro como herramienta para resolver relaciones de recurrencia de la forma

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n),$$

indicar los casos de análisis que contempla según la comparación entre $f(n)$ y $n^{\log_b a}$, y proponer tres casos de uso, planteando la relación de recurrencia de cada uno y resolviéndola con el teorema.

2.2. Relaciones de recurrencia y divide y vencerás

Muchos algoritmos siguen el paradigma de *divide y vencerás*: dividen el problema en subproblemas más pequeños, los resuelven de forma recursiva y combinan sus soluciones. El tiempo de ejecución de estos algoritmos se expresa de manera natural mediante una **relación de recurrencia**, es decir, una ecuación que define $T(n)$ en términos del costo sobre entradas más pequeñas. El Teorema Maestro proporciona una solución asintótica directa para las recurrencias de este tipo sin necesidad de desarrollarlas manualmente (**?**).

2.3. Forma general de la recurrencia

El Teorema Maestro se aplica a recurrencias de la forma

$$T(n) = a\,T\left(\frac{n}{b}\right) + f(n), \qquad a \geq 1,\ b > 1,$$

donde:

- n es el tamaño del problema.
- a es el número de subproblemas en que se divide el problema en cada llamada recursiva (constante, $a \geq 1$).
- n/b es el tamaño de cada subproblema (constante $b > 1$). Se admite que sea $\lfloor n/b \rfloor$ o $\lceil n/b \rceil$ sin alterar el resultado asintótico.
- $f(n)$ es el costo de dividir el problema y combinar las soluciones de los subproblemas; debe ser una función asintóticamente positiva.
- Se asume un caso base $T(n) = \Theta(1)$ para n suficientemente pequeño.

2.4. Intuición: el árbol de recursión

Al desarrollar la recurrencia se obtiene un árbol de recursión con las siguientes propiedades:

- En el nivel i hay a^i subproblemas, cada uno de tamaño n/b^i, por lo que el costo total de ese nivel es $a^i f(n/b^i)$.
- La altura del árbol es $\log_b n$, ya que $n/b^i = 1$ cuando $i = \log_b n$.
- El número de hojas es $a^{\log_b n} = n^{\log_b a}$, y su costo total es $\Theta\big(n^{\log_b a}\big)$.

Por lo tanto,

$$T(n) = \Theta\big(n^{\log_b a}\big) + \sum_{i=0}^{\log_b n - 1} a^i \, f\Big(\frac{n}{b^i}\Big).$$

El primer término es el costo de las hojas y la suma es el costo de los nodos internos. El Teorema Maestro determina cuál de los dos domina comparando $f(n)$ con la **función crítica** $n^{\log_b a}$.

2.5. Enunciado del Teorema Maestro

Teorema (Maestro). Sean $a \geq 1$ y $b > 1$ constantes, sea $f(n)$ una función asintóticamente positiva y sea $T(n)$ definida para los enteros no negativos por la recurrencia $T(n) = a\,T(n/b) + f(n)$. Entonces $T(n)$ tiene las siguientes cotas asintóticas:

- **Caso 1 (domina el costo de las hojas).** Si $f(n) = O\big(n^{\log_b a - \varepsilon}\big)$ para alguna constante $\varepsilon > 0$, entonces
$$T(n) = \Theta\big(n^{\log_b a}\big).$$
- **Caso 2 (el costo se reparte entre todos los niveles).** Si $f(n) = \Theta\Big(n^{\log_b a} \log^k n\Big)$ para alguna constante $k \geq 0$, entonces
$$T(n) = \Theta\Big(n^{\log_b a} \log^{k+1} n\Big).$$
En particular, para $k = 0$: si $f(n) = \Theta\big(n^{\log_b a}\big)$, entonces $T(n) = \Theta\big(n^{\log_b a} \log n\big)$.
- **Caso 3 (domina el costo de la raíz).** Si $f(n) = \Omega\big(n^{\log_b a + \varepsilon}\big)$ para alguna constante $\varepsilon > 0$, y si además se cumple la *condición de regularidad*
$$a\,f\Big(\frac{n}{b}\Big) \leq c\,f(n) \quad \text{para alguna constante } c < 1 \text{ y } n \text{ suficientemente grande,}$$
entonces
$$T(n) = \Theta(f(n)).$$

2.5.1. Resumen de los casos

Caso	Comparación de $f(n)$ con $n^{\log_b a}$	Solución
1	$f(n)$ crece polinomialmente más lento	$\Theta\big(n^{\log_b a}\big)$
2	$f(n)$ crece al mismo ritmo (salvo factores $\log^k n$)	$\Theta\Big(n^{\log_b a} \log^{k+1} n\Big)$
3	$f(n)$ crece polinomialmente más rápido (y es regular)	$\Theta(f(n))$

2.6. Procedimiento para aplicar el teorema

- **Paso 1:** identificar los valores de a, b y $f(n)$ en la recurrencia.
- **Paso 2:** calcular la función crítica $n^{\log_b a}$.
- **Paso 3:** comparar $f(n)$ con $n^{\log_b a}$ y determinar en cuál de los tres casos se encuentra.
- **Paso 4:** si es el caso 3, verificar la condición de regularidad.
- **Paso 5:** escribir la solución asintótica correspondiente.

2.7. Limitaciones del teorema

El Teorema Maestro no cubre todas las recurrencias de divide y vencerás. No es aplicable, entre otros, en los siguientes casos:

- Cuando $f(n)$ es menor que $n^{\log_b a}$ pero no de forma polinomial (existe una brecha entre los casos 1 y 2). Por ejemplo, $T(n) = 2T(n/2) + n/\log n$: aquí $n^{\log_2 2} = n$, y $f(n)$ no es $O(n^{1-\varepsilon})$ para ningún $\varepsilon > 0$, ni tampoco $\Theta(n \log^k n)$ con $k \geq 0$.
- Cuando $f(n)$ es mayor que $n^{\log_b a}$ pero no de forma polinomial, o cuando no cumple la condición de regularidad (brecha entre los casos 2 y 3).
- Cuando a no es constante o es menor que 1, o cuando los subproblemas no tienen el mismo tamaño (por ejemplo, $T(n) = T(n/3) + T(2n/3) + n$).
- Cuando la recurrencia no es de la forma $T(n/b)$, por ejemplo las recurrencias por sustracción como $T(n) = T(n-1) + n$.

En esos casos se recurre a otros métodos: sustitución, árbol de recursión o el método de Akra–Bazzi (**?**), que generaliza el Teorema Maestro. La versión del teorema con divisiones de la forma $a\,T(n/b) + f(n)$ se remonta al trabajo de Bentley, Haken y Saxe (**?**).

2.8. Tres casos de uso

2.8.1. Caso de uso 1: Búsqueda binaria

La búsqueda binaria compara el elemento buscado con el elemento central de un arreglo ordenado y continúa en una sola de las dos mitades. La comparación y el cálculo del punto medio cuestan tiempo constante.

Relación de recurrencia:

$$T(n) = T\Big(\frac{n}{2}\Big) + \Theta(1).$$

Resolución con el Teorema Maestro:

- Parámetros: $a = 1$, $b = 2$, $f(n) = \Theta(1)$.
- Función crítica: $n^{\log_2 1} = n^0 = 1$.
- Comparación: $f(n) = \Theta(1) = \Theta\big(n^0 \log^0 n\big)$, por lo que se cumple el **caso 2** con $k = 0$.
- Solución:

$$T(n) = \Theta\big(n^0 \log n\big) = \Theta(\log n).$$

2.8.2. Caso de uso 2: Merge Sort

Merge Sort divide el arreglo en dos mitades, ordena cada una de forma recursiva y las mezcla en tiempo lineal.

Relación de recurrencia:

$$T(n) = 2\,T\left(\frac{n}{2}\right) + \Theta(n).$$

Resolución con el Teorema Maestro:

- Parámetros: $a = 2$, $b = 2$, $f(n) = \Theta(n)$.
- Función crítica: $n^{\log_2 2} = n$.
- Comparación: $f(n) = \Theta(n) = \Theta\left(n^{\log_2 2}\right)$, por lo que se cumple el **caso 2** con $k = 0$.
- Solución:

$$T(n) = \Theta(n \log n).$$

2.8.3. Caso de uso 3: Multiplicación de Karatsuba

El algoritmo de Karatsuba multiplica dos números de n dígitos reduciendo las cuatro multiplicaciones de tamaño $n/2$ del método clásico a solamente tres, a costa de sumas y restas adicionales de costo lineal.

Relación de recurrencia:

$$T(n) = 3\,T\left(\frac{n}{2}\right) + \Theta(n).$$

Resolución con el Teorema Maestro:

- Parámetros: $a = 3$, $b = 2$, $f(n) = \Theta(n)$.
- Función crítica: $n^{\log_2 3} \approx n^{1{,}585}$.
- Comparación: $f(n) = n = O\left(n^{\log_2 3 - \varepsilon}\right)$ con, por ejemplo, $\varepsilon = 0{,}5$, ya que $n^{1{,}085}$ crece más rápido que n. Se cumple el **caso 1**.
- Solución:

$$T(n) = \Theta\left(n^{\log_2 3}\right) \approx \Theta\left(n^{1{,}585}\right),$$

que mejora asintóticamente al $\Theta(n^2)$ del método clásico.

2.9. Ejemplo adicional: el caso 3

Los tres casos de uso anteriores ilustran los casos 1 y 2. Para el caso 3 se puede considerar la selección de un elemento por orden (*Quickselect*) en su escenario favorable, donde cada partición divide el arreglo en mitades y solo se continúa en una de ellas. La partición cuesta tiempo lineal.

Relación de recurrencia:

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(n).$$

Resolución con el Teorema Maestro:

- Parámetros: $a = 1$, $b = 2$, $f(n) = \Theta(n)$.
- Función crítica: $n^{\log_2 1} = 1$.
- Comparación: $f(n) = n = \Omega\left(n^{0+\varepsilon}\right)$ con $\varepsilon = 1$, por lo que corresponde al **caso 3**.
- Regularidad: $a\,f(n/b) = \dfrac{n}{2} \le \dfrac{1}{2}\,n$, y se cumple con $c = 1/2 < 1$.

- Solución:

$$T(n) = \Theta(f(n)) = \Theta(n).$$

Otro ejemplo que muestra la versión extendida del caso 2 es $T(n) = 2T(n/2)+n\log n$: con $a = b = 2$ se tiene $f(n) = \Theta(n\log^1 n)$, es decir $k = 1$, y por tanto $T(n) = \Theta(n\log^2 n)$.

2.10. Resumen de los casos de uso

Algoritmo	**Recurrencia**	$n^{\log_b a}$	**Caso**	**Solución**
Búsqueda binaria	$T(n) = T(n/2) + \Theta(1)$	1	2	$\Theta(\log n)$
Merge Sort	$T(n) = 2T(n/2) + \Theta(n)$	n	2	$\Theta(n\log n)$
Karatsuba	$T(n) = 3T(n/2) + \Theta(n)$	$n^{1,585}$	1	$\Theta(n^{1,585})$
Quickselect (favorable)	$T(n) = T(n/2) + \Theta(n)$	1	3	$\Theta(n)$

3. Relaciones de Recurrencia

3.1. Objetivo

Investigar las relaciones de recurrencia como herramienta para describir el costo de los algoritmos recursivos, presentar los métodos para resolverlas (iteración, sustitución, árbol de recursión, ecuación característica y Teorema Maestro) y desarrollar casos de uso planteando y resolviendo la recurrencia de cada uno.

3.2. Definición

Una **relación de recurrencia** es una ecuación que define cada término de una sucesión $a_0, a_1, a_2, \ldots$ en función de uno o más términos anteriores. Para que la sucesión quede completamente determinada, la relación debe acompañarse de **condiciones iniciales** (o **caso base**) (**?**).

Ejemplos clásicos:

- Factorial: $n! = n \cdot (n-1)!$, con $0! = 1$.
- Sucesión de Fibonacci: $F_n = F_{n-1} + F_{n-2}$, con $F_0 = 0$ y $F_1 = 1$.
- Torres de Hanoi: $H_n = 2H_{n-1} + 1$, con $H_0 = 0$.

Resolver una relación de recurrencia significa encontrar una *fórmula cerrada* para a_n que no dependa de términos anteriores, o al menos una cota asintótica (O, Ω, Θ) de su crecimiento.

3.3. Recurrencias en el análisis de algoritmos

Si $T(n)$ es el tiempo de ejecución de un algoritmo recursivo sobre una entrada de tamaño n, la recurrencia se plantea a partir de la estructura del algoritmo (**?**):

- **Caso base:** el costo cuando la entrada es lo bastante pequeña para resolverse sin recursión (normalmente $T(1) = \Theta(1)$).
- **Llamadas recursivas:** cuántas se realizan y sobre qué tamaño de entrada, lo que produce términos como $T(n-1)$ o $aT(n/b)$.
- **Costo no recursivo:** el trabajo que se hace fuera de las llamadas (dividir, combinar, comparar), expresado como $f(n)$.

Con estos elementos, la recurrencia tiene la forma $T(n) = (\text{costo de las llamadas recursivas}) + f(n)$.

3.4. Clasificación de las recurrencias

- **Según el orden:** el orden es la distancia entre el término más reciente y el más antiguo que aparece. $a_n = 2a_{n-1} + 1$ es de orden 1; $F_n = F_{n-1} + F_{n-2}$ es de orden 2.
- **Lineales y no lineales:** son lineales si los términos previos aparecen solo multiplicados por coeficientes y sumados. Ejemplo no lineal: $a_n = a_{n-1}^2 + 1$.
- **Homogéneas y no homogéneas:** una recurrencia lineal es homogénea si no tiene términos independientes de la sucesión (por ejemplo $F_n = F_{n-1} + F_{n-2}$), y no homogénea en caso contrario (por ejemplo $H_n = 2H_{n-1} + 1$).
- **Coeficientes constantes o variables:** los coeficientes que multiplican a los términos previos son constantes, como en Fibonacci, o dependen de n.
- **Por sustracción o por división:** en las primeras el tamaño disminuye restando una constante, $T(n) = aT(n-c) + f(n)$; en las segundas se divide entre una constante, $T(n) = aT(n/b) + f(n)$ (divide y vencerás).

3.5. Métodos de resolución

3.5.1. Método de iteración (sustitución hacia atrás)

Consiste en *desenrollar* la recurrencia sustituyendo repetidamente el término anterior hasta detectar un patrón en la k-ésima sustitución y luego elegir k para llegar al caso base.

- **Ventaja:** no requiere adivinar la solución.
- **Limitación:** el álgebra puede volverse complicada y el patrón general debe demostrarse (normalmente por inducción).

3.5.2. Método de sustitución (adivinar y demostrar)

- **Paso 1:** adivinar la forma de la solución (por ejemplo, $T(n) \leq c\,n \log n$).
- **Paso 2:** demostrar por inducción que la cota se cumple, suponiendo que vale para tamaños menores.
- **Paso 3:** elegir las constantes de forma que también se cumpla el caso base.

3.5.3. Método del árbol de recursión

Se dibuja el árbol de llamadas recursivas, se calcula el costo de cada nivel y se suman los costos de todos los niveles. Sirve para generar una buena conjetura que luego puede confirmarse con el método de sustitución.

3.5.4. Ecuación característica (recurrencias lineales homogéneas de coeficientes constantes)

Dada la recurrencia

$$a_n = c_1 a_{n-1} + c_2 a_{n-2} + \cdots + c_k a_{n-k},$$

su **ecuación característica** es

$$r^k - c_1 r^{k-1} - c_2 r^{k-2} - \cdots - c_k = 0.$$

- **Raíces distintas** $r_1, \ldots, r_k$: la solución general es

$$a_n = \alpha_1 r_1^n + \alpha_2 r_2^n + \cdots + \alpha_k r_k^n.$$

- **Raíz repetida** r de multiplicidad m: aporta a la solución el término

$$\left(\alpha_0 + \alpha_1 n + \cdots + \alpha_{m-1} n^{m-1}\right) r^n.$$

- Las constantes α_i se determinan resolviendo el sistema que resulta de aplicar las condiciones iniciales.
- **Caso no homogéneo:** la solución es la suma de la solución general de la parte homogénea y de una solución particular de la recurrencia completa. Las constantes se ajustan al final con las condiciones iniciales.

3.5.5. Teorema Maestro

Para recurrencias por división de la forma $T(n) = aT(n/b) + f(n)$ con $a \geq 1$ y $b > 1$, el Teorema Maestro da directamente la cota asintótica comparando $f(n)$ con $n^{\log_b a}$ (véase la sección anterior).

3.5.6. Otros métodos

Las *funciones generadoras* permiten resolver recurrencias que las técnicas anteriores no cubren, y el método de *Akra–Bazzi* generaliza el Teorema Maestro a subproblemas de tamaños distintos.

3.6. Casos de uso

3.6.1. Caso de uso 1: Torres de Hanoi (recurrencia por sustracción)

Para mover n discos de una varilla a otra, se mueven $n-1$ discos a la varilla auxiliar, se mueve el disco mayor y se mueven de nuevo los $n-1$ discos sobre él (**?**).

Relación de recurrencia:

$$H(n) = 2\,H(n-1) + 1, \qquad H(0) = 0.$$

Resolución por iteración:

$$\begin{aligned} H(n) &= 2H(n-1) + 1 \\ &= 2\big(2H(n-2) + 1\big) + 1 = 2^2 H(n-2) + 2 + 1 \\ &= 2^3 H(n-3) + 2^2 + 2 + 1 \\ &\;\;\vdots \\ &= 2^k H(n-k) + \big(2^{k-1} + \cdots + 2 + 1\big) = 2^k H(n-k) + 2^k - 1. \end{aligned}$$

Con $k = n$ se llega al caso base $H(0) = 0$:

$$H(n) = 2^n \cdot 0 + 2^n - 1 = 2^n - 1 = \Theta(2^n).$$

Verificación: $H(1) = 1$, $H(2) = 3$, $H(3) = 7$, que coinciden con $2^n - 1$.

Resolución alternativa (recurrencia no homogénea): la parte homogénea $H(n) = 2H(n-1)$ tiene como solución $\alpha 2^n$. Una solución particular constante p debe cumplir $p = 2p + 1$, es decir $p = -1$. Entonces $H(n) = \alpha 2^n - 1$, y con $H(0) = 0$ se obtiene $\alpha = 1$, lo que confirma $H(n) = 2^n - 1$.

3.6.2. Caso de uso 2: Sucesión de Fibonacci (ecuación característica)

Relación de recurrencia:

$$F(n) = F(n-1) + F(n-2), \qquad F(0) = 0,\ F(1) = 1.$$

Resolución con la ecuación característica:

- Ecuación característica: $r^2 - r - 1 = 0$.
- Raíces:
$$r_1 = \varphi = \frac{1+\sqrt{5}}{2} \approx 1{,}618, \qquad r_2 = \psi = \frac{1-\sqrt{5}}{2} \approx -0{,}618.$$
- Solución general: $F(n) = \alpha\,\varphi^{\,n} + \beta\,\psi^{\,n}$.
- Condiciones iniciales: de $F(0) = 0$ resulta $\alpha + \beta = 0$, y de $F(1) = 1$ resulta $\alpha\varphi + \beta\psi = 1$. Resolviendo, $\alpha = \dfrac{1}{\sqrt{5}}$ y $\beta = -\dfrac{1}{\sqrt{5}}$.
- Fórmula cerrada (fórmula de Binet):
$$F(n) = \frac{\varphi^{\,n} - \psi^{\,n}}{\sqrt{5}}.$$

Como $|\psi| < 1$, el segundo término tiende a 0 y $F(n) = \Theta(\varphi^{\,n})$. Por eso el algoritmo recursivo ingenuo, cuyo tiempo satisface la misma recurrencia más un costo constante, tiene crecimiento exponencial.

3.6.3. Caso de uso 3: Merge Sort (recurrencia por división)

Merge Sort divide el arreglo en dos mitades, las ordena recursivamente y las mezcla en tiempo lineal.

Relación de recurrencia:

$$T(n) = 2\,T\!\left(\frac{n}{2}\right) + c\,n, \qquad T(1) = \Theta(1).$$

Resolución por iteración (con $n = 2^k$):

$$\begin{aligned} T(n) &= 2T(n/2) + cn \\ &= 2\big(2T(n/4) + cn/2\big) + cn = 4T(n/4) + 2cn \\ &= 8T(n/8) + 3cn \\ &\ \ \vdots \\ &= 2^k T(n/2^k) + k\,c\,n. \end{aligned}$$

Con $k = \log_2 n$ se llega a $T(1)$:

$$T(n) = n\,T(1) + c\,n\log_2 n = \Theta(n \log n).$$

Comprobación con el Teorema Maestro: $a = b = 2$, $n^{\log_2 2} = n = \Theta(f(n))$, es decir, caso 2 con $k = 0$, que también da $\Theta(n \log n)$.

3.7. Ejemplos adicionales por método

3.7.1. Método de sustitución

Sea $T(n) = 2T(\lfloor n/2 \rfloor) + n$ con $T(1) = 1$. Se conjetura $T(n) \leq c\, n \log_2 n$ para $n \geq 2$.

- **Paso inductivo:** suponiendo la cota para $\lfloor n/2 \rfloor$,

$$\begin{aligned} T(n) &\leq 2c \left\lfloor \frac{n}{2} \right\rfloor \log_2 \left\lfloor \frac{n}{2} \right\rfloor + n \leq c\, n \log_2 \frac{n}{2} + n \\ &= c\, n \log_2 n - c\, n + n \leq c\, n \log_2 n \quad \text{si } c \geq 1. \end{aligned}$$

- **Casos base:** se toman $n = 2$ y $n = 3$, ya que para $n = 1$ se tiene $\log_2 1 = 0$ y la cota fallaría. Como $T(2) = 4$ y $T(3) = 5$, basta $c = 2$.
- **Conclusión:** $T(n) = O(n \log n)$.

3.7.2. Método del árbol de recursión

Sea $T(n) = 3T(n/4) + c\, n^2$.

- En el nivel i hay 3^i nodos, cada uno de costo $c\,(n/4^i)^2$, por lo que el costo del nivel es $\left(\frac{3}{16}\right)^i c\, n^2$.
- La altura es $\log_4 n$ y el número de hojas es $3^{\log_4 n} = n^{\log_4 3}$.
- La suma de los niveles internos es una serie geométrica decreciente:

$$\sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i c\, n^2 < c\, n^2 \sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i = \frac{16}{13}\, c\, n^2.$$

- El costo de las hojas es $\Theta\left(n^{\log_4 3}\right) = o(n^2)$, y el costo de la raíz ya es $c\, n^2$. Por tanto, $T(n) = \Theta(n^2)$.

Este resultado coincide con el caso 3 del Teorema Maestro ($a = 3$, $b = 4$, $n^{\log_4 3} \approx n^{0{,}79}$, y la regularidad se cumple porque $3(n/4)^2 = \frac{3}{16}\, n^2$, es decir, con $c = 3/16 < 1$).

3.7.3. Ecuación característica con raíces distintas

Sea $a_n = 5a_{n-1} - 6a_{n-2}$ con $a_0 = 2$ y $a_1 = 5$.

- Ecuación característica: $r^2 - 5r + 6 = 0$, con raíces $r = 2$ y $r = 3$.
- Solución general: $a_n = \alpha\, 2^n + \beta\, 3^n$.
- Condiciones iniciales: $\alpha + \beta = 2$ y $2\alpha + 3\beta = 5$, de donde $\alpha = \beta = 1$.
- Solución: $a_n = 2^n + 3^n$. Verificación: $a_2 = 5 \cdot 5 - 6 \cdot 2 = 13 = 2^2 + 3^2$.

3.7.4. Ecuación característica con raíz repetida

Sea $a_n = 4a_{n-1} - 4a_{n-2}$ con $a_0 = 1$ y $a_1 = 4$.

- Ecuación característica: $r^2 - 4r + 4 = (r - 2)^2 = 0$, con raíz doble $r = 2$.
- Solución general: $a_n = (\alpha + \beta n)\, 2^n$.
- Condiciones iniciales: $\alpha = 1$ y $(\alpha + \beta) \cdot 2 = 4$, de donde $\beta = 1$.
- Solución: $a_n = (n + 1)\, 2^n$. Verificación: $a_2 = 4 \cdot 4 - 4 \cdot 1 = 12 = 3 \cdot 2^2$.

3.8. Resumen

Recurrencia	**Método habitual**	**Solución**
$T(n) = T(n-1) + c$	Iteración	$\Theta(n)$
$T(n) = T(n-1) + n$	Iteración	$\Theta(n^2)$
$T(n) = 2T(n-1) + 1$	Iteración / no homogénea	$\Theta(2^n)$
$T(n) = T(n-1) + T(n-2)$	Ecuación característica	$\Theta(\varphi^n)$
$T(n) = T(n/2) + c$	Iteración / Teorema Maestro	$\Theta(\log n)$
$T(n) = 2T(n/2) + n$	Iteración / Teorema Maestro	$\Theta(n \log n)$
$T(n) = 3T(n/4) + n^2$	Árbol / Teorema Maestro	$\Theta(n^2)$

Cuándo usar cada método:

- **Iteración:** recurrencias simples por sustracción o por división cuando se puede detectar el patrón.
- **Árbol de recursión:** para obtener una conjetura o cuando el costo por nivel forma una serie reconocible.
- **Sustitución:** para demostrar formalmente una cota conjeturada.
- **Ecuación característica:** recurrencias lineales de coeficientes constantes, como Fibonacci.
- **Teorema Maestro:** recurrencias de la forma $aT(n/b) + f(n)$ que cumplan sus hipótesis.